

Contents

NaviSense AI — Technical Report	1
1. Target Application	1
2. System Architecture	1
3. Datasets	2
4. AMD Radeon GPU & ROCm Utilization	3
5. Innovations & Key Technical Contributions	3
6. Deliverables	4
7. Additional Strengths	4
8. Team	4

NaviSense AI — Technical Report

Embodied SMCP Training Environment AMD AI DevMaster Hackathon — Track 3: Physical AI Team: EZi Edutech

- Live demo: <https://navisense.eziedutech.dev>
- Source code: <https://github.com/ziaac/navisenseAI>
- Demo video (3 min): <https://youtu.be/j2lzzsKH6QE>

1. Target Application

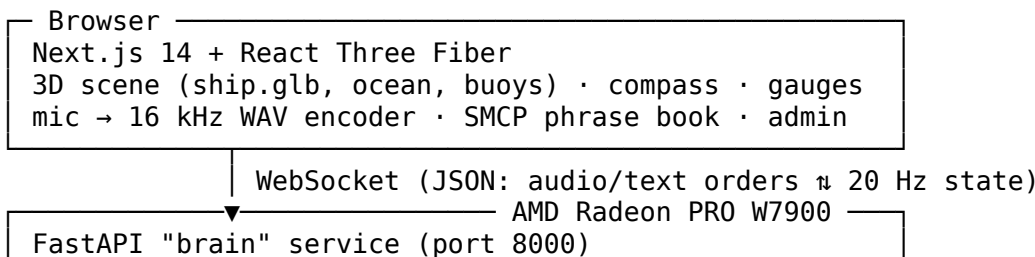
NaviSense AI is an embodied-AI training environment for maritime cadets learning the **IMO Standard Marine Communication Phrases (SMCP)** — the international standard English used for helm and engine orders at sea. Communication failure is a recognized contributing factor in maritime accidents, and vocational maritime schools (particularly in Indonesia, one of the world’s largest seafarer-supplying countries) have limited access to full-mission bridge simulators, which cost hundreds of thousands of dollars.

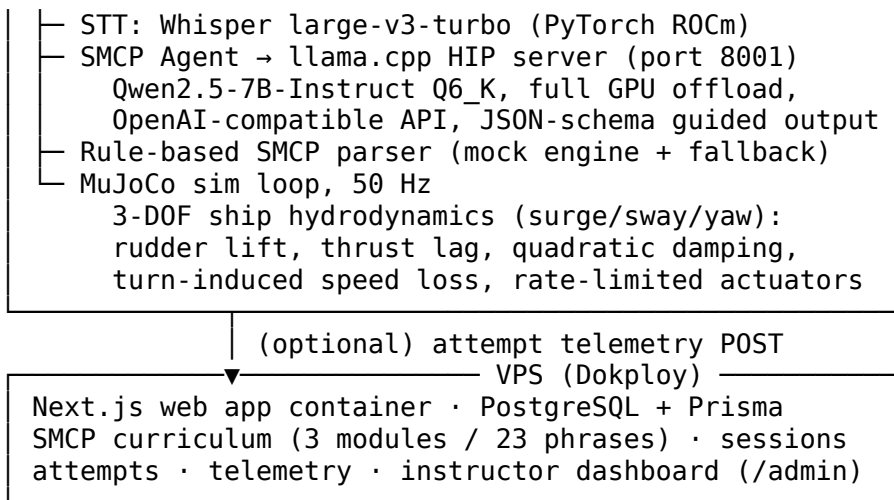
NaviSense AI closes this gap with a browser-accessible bridge simulator driven entirely by voice. A cadet speaks a helm/engine order (e.g., “*Hard-a-port, ahead dead slow*”). The system:

1. **Transcribes** the utterance locally (Whisper on AMD GPU),
2. **Validates** it linguistically against the SMCP syllabus using a local LLM with schema-constrained JSON output,
3. **Executes** the resulting actuation parameters (rudder angle, engine telegraph) on a physics-simulated ship (MuJoCo rigid-body dynamics with 3-DOF ship hydrodynamics),
4. **Renders** the ship’s response in a real-time 3D scene and bridge instrument panel, and returns corrective linguistic feedback.

The embodiment loop — *speech* → *cognition* → *physical actuation* → *visual/instrument feedback* — is what turns language drill into embodied learning: the cadet immediately sees the physical consequence of a correctly (or incorrectly) phrased order.

2. System Architecture





Deployment topology. The web app and database run in Docker containers on a VPS (Dokploy/Traefik, HTTPS). The brain service runs on the AMD Radeon Cloud GPU instance and is exposed via a Cloudflare named tunnel (`wss://navisense-gpu.eziedutech.dev`). The UI offers two engines with automatic failover: **AI MODE** (full GPU pipeline) and **MOCK MODE** (deterministic rule-based SMCP parser, CPU-only) — the same parser also serves as an automatic fallback if the LLM endpoint becomes unreachable, so a live demo can never dead-end.

Physics model. The ship is a free-floating rigid body in MuJoCo (timestep 20 ms, RK4). Each step, a 3-DOF maneuvering model (Fossen/Nomoto-style simplification) computes body-frame forces: propeller thrust with first-order engine lag ($\tau = 8$ s), rudder lift proportional to flow² with a realistic steering-gear rate limit (5°/s), linear+quadratic damping in surge/sway/yaw, and turn-induced added resistance. The result reproduces characteristic full-scale behavior: ~18.5 kn cruise at Full Ahead, a stable ~1.2°/s turn under hard rudder with speed bleeding to ~8.6 kn — the speed-loss-in-turn phenomenon cadets must anticipate in real ship handling.

Grounded language-to-action contract. The LLM is forced to emit schema-valid JSON (`smcp_valid`, `linguistic_feedback`, `physics_action{rudder_angle_deg, engine_thrust_pct}`) via guided decoding. The agent negotiates three JSON-constraint dialects (`json_schema` → `llama.cpp's json_object+schema` → `plain json_object`) so it runs unmodified against `llama.cpp`, `vLLM`, or any OpenAI-compatible server. Outputs are defensively clamped to actuator limits before reaching the simulator.

3. Datasets

- **SMCP curriculum dataset (self-built):** 23 canonical helm/engine/combined phrases from IMO SMCP A1/6.2, organized into 3 modules and stored relationally (PostgreSQL/Prisma) with expected actuation ground truth per phrase. Used for cadet evaluation and as the reference the agent validates against.
- **Voice evaluation set (self-built):** recorded spoken orders (native/non-native accents) used to validate the Whisper → LLM → actuation pipeline end-to-end in production.
- **Pre-trained open models (no training performed):** Qwen2.5-7B-Instruct (Apache-2.0, GGUF Q6_K quantization by bartowski) and Whisper large-v3-turbo (MIT). The linguistic-validation task is solved zero-shot via prompt engineering + schema-guided decoding, keeping the entire system reproducible on a single GPU with no fine-tuning cost.
- **3D asset:** container-ship model (FBX) converted through an automated Blender-as-module (bpy) pipeline to a 1.13 MB Draco+WebP glTF binary (188,711 triangles, 3 PBR materials).

4. AMD Radeon GPU & ROCm Utilization

All AI inference runs **locally on a single AMD Radeon PRO W7900 (48 GB, gfx1100, ROCm 7.2.1)** on Radeon Cloud. No remote AI APIs are used at any stage.

Stage	Backend	ROCm path
Speech-to-text	Whisper large-v3-turbo	PyTorch 2.7.1 + rocm6.3 wheels (HIP)
SMCP validation LLM	Qwen2.5-7B-Instruct Q6_K	llama-cpp-python 0.3.34 compiled with -DGGML_HIP=ON -DAMDGPU_TARGETS=gfx1100, all layers offloaded to GPU
Physics	MuJoCo 50 Hz	CPU (GPU headroom reserved)

Measured performance (production, 2026-07-19):

Metric	Value
LLM generation throughput	89-90 tokens/s (11.1 ms/token)
LLM prompt eval	289-545 tokens/s (cached prefix)
Text order end-to-end latency	1.0-1.5 s
Voice order end-to-end latency	1.5-2.5 s
Whisper (warm, 3 s audio)	~0.06 s
Physics step budget	27 ms per order application (50 Hz loop)
VRAM in use (LLM+Whisper resident)	24 % (~11.5 GB of 48 GB)
GPU temperature / idle power	27 °C / 12-14 W

Engineering notes specific to AMD/ROCm that we documented and reproduce in the README: vLLM PyPI wheels are CUDA-only, so we compiled llama.cpp with the HIP backend ($\approx 5\text{--}10$ min on the instance's 128 cores) — an approach that keeps an OpenAI-compatible, schema-guided serving layer fully on ROCm; PyTorch ROCm wheels bundle their own userland and run cleanly on the ROCm 7.2.1 host; 76 % VRAM headroom remains for planned extensions (larger models, YOLO-based visual perception).

5. Innovations & Key Technical Contributions

1. **Language-embodied training loop.** To our knowledge the first open-source trainer that couples IMO SMCP vocational-English validation directly to physics actuation: correctness of *language* is experienced as ship *behavior*.
2. **Schema-grounded LLM actuation with dialect negotiation.** A defensive guided-JSON contract that works across OpenAI-compatible servers (llama.cpp/vLLM), with hard clamps and a “keep current state on invalid order” semantic — the LLM cannot produce out-of-envelope actuation.
3. **Dual-engine resilience.** A deterministic SMCP parser (IMO A1/6.2 grammar) that is simultaneously the CPU demo engine, the automatic LLM fallback, and a development tool — the full product loop runs with or without the GPU, and the UI fails over transparently.
4. **Compact but honest ship physics.** A tuned 3-DOF hydrodynamic model on MuJoCo reproducing rate-limited steering gear, engine telegraph lag, and turn speed-loss — the behaviors that matter pedagogically for helm orders — at 50 Hz with real-time streaming to the browser.
5. **Fully reproducible single-GPU deployment,** documented step-by-step (environment, model download via mirror, HIP compile flags, service scripts), with Docker images for every non-GPU component and a runbook for the GPU instance.

6. Deliverables

Deliverable	Form
Live production demo	https://navisense.eziedutech.dev (web + DB on VPS, AI engine on Radeon Cloud via wss://navisense-gpu.eziedutech.dev)
Source code	https://github.com/ziaac/navisenseAI (MIT) — web app, brain service, physics, curriculum seed, Dockerfiles
Docker images	<code>docker/Dockerfile.web</code> (Next.js+Prisma), <code>docker/Dockerfile.brain-cpu</code> (mock engine), <code>docker/Dockerfile.rocm</code> (GPU brain); published images per README
Reproducibility README	Repository root — GPU setup, CPU quickstart, web+DB, step-by-step reproduction
Demo video	https://youtu.be/j2lzzsKH6QE (3 min: voice orders, AI vs mock engines, physics response, instruments, admin)
Instructor dashboard	<code>/admin</code> — curriculum, sessions, attempt telemetry, validity rate
Upstream contribution	<code>mujoco-ship-hydro</code> — standalone open-source MuJoCo ship-hydrodynamics example with AMD/ROCm serving notes (see repository)

7. Additional Strengths

- **Real-world adoption path:** built by an education-technology team for Indonesian maritime vocational schools (SMK Pelayaran); the curriculum model, attempt telemetry, and instructor dashboard are the foundation of an LMS, not just a demo.
- **Low-latency local-first design:** every AI stage is local; the only network hop is the WebSocket to the browser.
- **Production hardening:** automatic engine failover, actuator clamping, WS auto-reconnect, telemetry, HTTPS/WSS end-to-end.
- **Honest resource profile:** the entire pipeline uses ~24 % of one W7900 — demonstrating that meaningful embodied-AI education tools fit comfortably on a single Radeon GPU.

8. Team

EZi Edutech — solo project.

Member	Role & contribution
Zia (GitHub: ziaac)	Concept & maritime curriculum design; full-stack development (physics simulation, brain service, LLM/STT integration on ROCm, web frontend, database); deployment & operations (Radeon Cloud GPU instance, VPS, tunnels); documentation and demo video.